

Sprint 11 Structured Notes: Relational Databases and PostgreSQL Setup

1. Current Progress

You are now moving from the **Git section** into the **database section** of the Relational Databases learning path.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance

So far, you completed these stages:

Sprint 1: Tools and VS Code

You learned about:

- code editors and IDEs
- VS Code
- workspace
- project folders
- Live Server
- basic shortcuts and extensions

Simple meaning: Sprint 1 prepared your coding environment.

Sprint 2: Terminal, Shell, and Command Line Basics

You learned about:

- terminal
- command line
- shell
- Bash
- command history
- auto-complete
- clearing and interrupting commands

Simple meaning: Sprint 2 taught you what happens when you type commands.

Sprint 3: Bash File and Folder Commands

You learned how to:

- move between folders
- create files and folders
- rename, copy, and delete files
- read files
- use flags and options

Simple meaning: Sprint 3 taught you how to control files and folders from the terminal.

Sprint 4: Bash Scripting Basics

You learned about:

- Bash scripts
- `.sh` files
- shebang
- interpreter paths
- variables
- arrays
- loops
- `do` and `done`
- variable interpolation

Simple meaning: Sprint 4 taught you how to save terminal commands into reusable script files.

Sprint 5 to Sprint 10: Git Foundation

You learned:

- Git setup
- local Git workflow
- GitHub repositories and remotes
- branches and pull requests
- forks and sync
- `.gitignore`
- merge conflicts
- safer Git recovery tools
- beginner CI/CD ideas

Simple meaning: Sprints 5 to 10 taught you how developers track, share, review, and protect project changes.

2. What Sprint 11 Is About

Sprint 11 is the **first PostgreSQL and database foundation sprint**.

Now you start learning:

- what a relational database is
- how data is stored in tables
- how tables connect to each other
- what PostgreSQL is
- how to open PostgreSQL tools like **pgAdmin** and **psql**

Simple meaning: Sprint 11 teaches you the basic ideas behind databases before you start writing real SQL in Sprint 12.

3. Main Goal of Sprint 11

The main goal of Sprint 11 is:

Understand relational databases and set up PostgreSQL tools.

By the end of Sprint 11, you should be able to:

- explain what a relational database is
- explain table, row, and column
- explain record, attribute, and field
- explain what a schema is
- understand the basic idea of primary keys and foreign keys
- explain relational vs non-relational databases
- explain what PostgreSQL is
- explain what pgAdmin is
- explain what psql is
- understand common PostgreSQL installation ideas
- connect to the default `postgres` database
- recognize the `postgres=#` prompt

Simple meaning: Sprint 11 is about understanding database ideas and making sure PostgreSQL is ready to use.

4. Big Beginner Idea

A database is like a very organized storage system.

Imagine a school. The school needs to store:

- student details
- teacher details
- course details
- payment details
- exam results

Instead of putting everything into one messy file, a relational database stores data in **tables**.

Example:

students table

student_id	name	age
1	Amal	20
2	Nimal	21

courses table

course_id	course_name
1	Database Systems
2	Web Development

Simple meaning: A relational database stores data in separate tables and connects related information.

5. What Is a Relational Database?

A **relational database** is a database that stores data in tables.

Each table usually stores one type of thing.

Examples:

- a **students** table stores student data
- a **courses** table stores course data
- a **customers** table stores customer data
- an **orders** table stores order data

A relational database connects related information using common attributes such as IDs.

Simple meaning: A relational database stores organized data in tables and can link those tables together.

6. What Is a Table?

A **table** is like a spreadsheet inside the database.

Example:

id	name	age
1	Gino	3
2	Nora	2

A table stores one category of information.

Examples of table names:

- students
- products
- customers
- orders
- dogs

Simple meaning: A table is a structured place to store one type of data.

7. What Is a Row?

A **row** is one full record in a table.

Example:

id	name	age
1	Gino	3

This full line is one row.

Rows are also called **records**.

Simple meaning: A row is one complete item in the table.

8. What Is a Column?

A **column** is one type of information in a table.

Example:

id	name	age
----	------	-----

The columns are:

- id
- name
- age

Columns are also called **fields** or **attributes**.

Simple meaning: A column is one kind of data inside the table.

9. Record, Attribute, and Field

These words are often used in database lessons.

Record

A **record** means one full row.

Example: 1, Gino, 3

Simple meaning: Record = row.

Attribute / Field

An **attribute** or **field** means one column.

Examples:

- name
- email
- price
- age

Simple meaning: Attribute = field = column.

10. Easy Memory Trick

Use this simple memory:

- **Table** = whole data box
- **Row** = one full record
- **Column** = one type of information

Example:

student_id	name	email
1	Amal	amal@email.com
2	Nimal	nimal@email.com

Here:

- `students` = table
 - Amal's full line = row / record
 - `email` = column / attribute / field
-

11. What Is a Schema?

A **schema** is the structure or design of the database.

A schema defines:

- what tables exist
- what columns each table has
- what data types those columns use
- what relationships exist between tables
- what rules or constraints apply

Example:

```
Database: school_db
```

```
Table: students
```

```
- student_id  
- name  
- email
```

```
Table: courses
```

- course_id
- course_name

This structure is part of the schema.

Simple meaning: A schema is the blueprint of the database.

12. What Is a Query?

A **query** is a request sent to the database.

Example request:

```
Show me all students.
```

Later in SQL, this may look like:

```
SELECT * FROM students;
```

Queries can be used to:

- add data
- change data
- delete data
- find data
- filter data
- sort data
- retrieve data
- summarize data

Simple meaning: A query is a question or instruction sent to the database.

13. What Is a Primary Key?

A **primary key** is a column that uniquely identifies each row in a table.

Example:

student_id	name
1	Amal

student_id	name
2	Nimal
3	Kamal

Here, `student_id` is the primary key.

A primary key:

- must be unique
- cannot be duplicated
- cannot be `NULL`
- identifies one exact row

Simple meaning: A primary key is the unique ID of a row.

Beginner examples:

- student ID identifies one student
- order ID identifies one order
- customer ID identifies one customer

14. What Is a Foreign Key?

A **foreign key** is a column that links one table to another table.

Example:

customers table

customer_id	name
1	Amal
2	Nimal

orders table

order_id	customer_id	item
101	1	Laptop
102	2	Mouse

In the `orders` table, `customer_id` points to the `customers` table.

Simple meaning: A foreign key connects one table to another table.

A foreign key helps prevent broken links between tables.

15. Relational vs Non-Relational Databases

Relational Databases

Relational databases:

- store data in tables
- use structured schemas
- support strong relationships between tables
- are good for organized business data

Common relational databases:

- PostgreSQL
- MySQL
- SQLite
- Microsoft SQL Server
- Oracle Database

Simple meaning: Relational databases are table-based and strongly organized.

Non-Relational Databases

Non-relational databases are also called **NoSQL databases**.

They store data more flexibly. They may not use strict connected tables.

They are often easier to change when the data structure changes often.

Simple meaning: Non-relational databases are more flexible and less rigid than relational databases.

Simple Comparison

Topic	Relational Database	Non-Relational Database
Structure	Tables	Flexible formats
Schema	Usually strict	Usually flexible
Relationships	Strong support	Depends on database type
Good for	Structured business data	Flexible or changing data

Simple meaning: Relational = more structured. Non-relational = more flexible.

16. Where Relational Databases Are Used

Relational databases are used in many areas, such as:

- healthcare
- construction
- business
- education
- gaming
- government
- e-commerce
- social media

Why are they used? Because they support:

- scalability
- data integrity
- organized storage
- linked data

Simple meaning: Relational databases are used when people need safe, organized, reliable data storage.

17. What Is PostgreSQL?

PostgreSQL is a relational database system.

It is known for being:

- free
- open source
- reliable
- strong for data integrity
- extensible

Simple meaning: PostgreSQL is professional database software used to store and manage relational data.

It is one of the main databases used in real projects.

18. PostgreSQL vs SQL

Many beginners confuse these two.

SQL

SQL is the language used to work with relational databases.

Example:

```
SELECT * FROM students;
```

PostgreSQL

PostgreSQL is the database system that understands SQL commands.

Easy Comparison

```
SQL = language  
PostgreSQL = database software
```

Another simple example:

```
English = language  
Teacher = person who understands English  
  
SQL = language  
PostgreSQL = database that understands SQL
```

Simple meaning: SQL is the language. PostgreSQL is the software.

19. Common Relational Database Systems

Some common relational database systems are:

- MySQL
- PostgreSQL
- SQLite
- Microsoft SQL Server
- Oracle Database

MySQL

Open source and popular in web development.

PostgreSQL

Free, open source, reliable, and strong for data integrity.

SQLite

Free, lightweight, file-based, self-contained, serverless, and zero-configuration.

Microsoft SQL Server

A Microsoft relational database system.

Oracle Database

Often used for large enterprise systems.

Simple meaning: PostgreSQL is one important member of the relational database family.

20. PostgreSQL Installation Overview

When installing PostgreSQL, you may see steps such as:

- opening the PostgreSQL website
- clicking the **Download** button
- choosing your operating system
- using the installer
- choosing the install directory
- choosing components
- choosing the data directory
- setting the superuser password
- confirming port 5432
- selecting locale
- reviewing the pre-installation summary
- seeing Stack Builder at the end

Simple meaning: PostgreSQL installation asks you where to install it, what tools to include, and how it should be configured.

21. Operating System Installation Idea

PostgreSQL installation can look different depending on your operating system.

Windows

Usually uses an installer.

macOS

Can use Homebrew.

Linux

May use package tools depending on the distribution.

For macOS Homebrew users, an example pattern is:

```
brew install postgresql@<version>
```

Simple meaning: The installation path changes by operating system, but the goal is the same: install PostgreSQL and its tools.

22. PostgreSQL Components

Common PostgreSQL components include:

- PostgreSQL Server
- pgAdmin 4
- Stack Builder
- Command Line Tools
- psql

Simple meaning: PostgreSQL installation gives you the main server plus tools to manage and use it.

23. PostgreSQL Server

The **PostgreSQL Server** is the main database engine.

It:

- stores the data
- runs the database logic
- accepts connections from tools like pgAdmin and psql

Simple meaning: PostgreSQL Server is the main program running the database.

If the server is not running, you cannot connect to the database.

24. pgAdmin

pgAdmin is a graphical administration and development tool for PostgreSQL.

It lets you manage PostgreSQL using a visual interface.

You can use it to look at:

- servers
- databases
- tables
- users
- settings
- query tools

Simple meaning: pgAdmin is like a visual control panel for PostgreSQL.

Easy Comparison

VS Code = visual tool for code
pgAdmin = visual tool for PostgreSQL databases

25. psql

psql is the terminal-based interactive tool for PostgreSQL.

You type commands into it.

Example:

```
psql -U postgres -d postgres
```

What this means

```
psql = open PostgreSQL terminal tool  
-U postgres = connect as user postgres  
-d postgres = connect to database postgres
```

Simple meaning: psql lets you talk to PostgreSQL from the terminal.

26. Stack Builder

Stack Builder is an optional tool that can install extra PostgreSQL tools and extensions.

Beginners usually do not need it deeply at first.

Simple meaning: Stack Builder helps install extra PostgreSQL add-ons.

27. Command Line Tools

Command Line Tools are terminal programs that come with PostgreSQL.

Examples:

- psql
- createdb
- dropdb

For Sprint 11, the main command-line tool you need is:

- psql

Simple meaning: Command Line Tools let you use PostgreSQL from the terminal.

28. Install Directory

The **install directory** is the folder where PostgreSQL program files are saved.

Simple meaning: Install directory = where PostgreSQL itself is installed.

29. Data Directory

The **data directory** is the folder where PostgreSQL stores actual database data.

Simple meaning: Data directory = where the database files live.

Important beginner rule: Do not randomly delete this folder.

30. Superuser Password

During installation, PostgreSQL usually asks you to set a password for the default database superuser.

Common username:

```
postgres
```

This password is important because you may need it when:

- opening pgAdmin
- opening psql
- connecting as the admin user

Simple meaning: The superuser password is the admin password for PostgreSQL.

Good beginner habit: Write it down safely.

31. Port 5432

PostgreSQL commonly uses port:

```
5432
```

A port is like a communication door for software.

Beginner example:

```
A building has many doors.  
A computer has many ports.  
PostgreSQL usually listens on door/port 5432.
```

Simple meaning: Port 5432 is the normal connection port for PostgreSQL.

32. Locale

Locale controls regional and language-related settings.

This may affect things like:

- text rules
- sorting
- formatting

Simple meaning: Locale tells PostgreSQL how to handle regional settings.

For beginners, the default locale is usually fine.

33. Pre-Installation Summary

Many installers show a **pre-installation summary** before the installation finishes.

This is where you check:

- install location
- selected components
- data directory
- port
- other main settings

Simple meaning: The pre-installation summary is your final review before PostgreSQL installs.

34. What Is the Default `postgres` Database?

After PostgreSQL is installed, there is usually a default database named:

```
postgres
```

This database exists so that you can connect and start working right away.

Simple meaning: `postgres` is the default starting database in PostgreSQL.

35. The `psql` Prompt

When you successfully connect to `psql`, you may see:

```
postgres=#
```

This prompt shows the connected database name.

Here, it shows you are connected to the `postgres` database.

Simple meaning: If you see `postgres=#`, you are inside PostgreSQL.

36. How to Open psql

The main command pattern is:

```
psql -U <username> -d <database_name>
```

Example:

```
psql -U postgres -d postgres
```

If it asks for a password, enter the PostgreSQL password you created during installation.

Expected prompt:

```
postgres=#
```

Simple meaning: This command opens PostgreSQL in the terminal.

37. What to Remember About psql

- `psql` is not pgAdmin
- `psql` is terminal-based
- it is interactive
- it lets you enter SQL commands and PostgreSQL shortcut commands
- it shows the connected database name in the prompt

Simple meaning: psql is the command-line door into PostgreSQL.

38. Sprint 11 Practice Task

Do this slowly.

Task A: Install or Confirm PostgreSQL

Check whether PostgreSQL is installed.

Try:

```
psql --version
```

If you see a version number, psql is installed.

Example:

```
psql (PostgreSQL) 16.x
```

Task B: Open pgAdmin

Open pgAdmin on your computer.

Check that you can see the server area. You may need your PostgreSQL password.

Task C: Open psql

Try:

```
psql -U postgres -d postgres
```

If your username is different, replace `postgres` with your real username.

Task D: Confirm the Prompt

You should try to reach this prompt:

```
postgres=#
```

That means you are connected.

Task E: Exit psql

To exit psql, type:

```
\q
```

Then press Enter.

Simple meaning: `\q` quits psql.

39. Important Sprint 11 Commands

```
psql --version
```

Checks whether psql is installed.

```
psql -U postgres -d postgres
```

Connects to the default `postgres` database as user `postgres`.

```
postgres=#
```

This is the psql prompt showing you are connected.

```
\q
```

Exits psql.

For macOS Homebrew users:

```
brew install postgresql@<version>
```

40. Why Sprint 11 Is Important

Sprint 11 is important because databases are at the center of many real applications.

Before learning SQL commands, you need to understand:

- what the database is
- how tables work

- how data connects
- what PostgreSQL tools do
- how to open and use those tools

Simple meaning: Sprint 11 gives you the database foundation that makes Sprint 12 and later sprints easier.

41. Common Beginner Mistakes

Mistake 1: Forgetting the PostgreSQL Password

Problem: You install PostgreSQL but forget the admin password.

Better habit: Write it down safely.

Simple meaning: You need the password to connect as the PostgreSQL admin user.

Mistake 2: Confusing pgAdmin and psql

Remember:

```
pgAdmin = visual tool  
psql = terminal tool
```

Both connect to PostgreSQL, but they are not the same.

Mistake 3: Thinking SQL and PostgreSQL Are the Same

Remember:

```
SQL = language  
PostgreSQL = database system
```

Mistake 4: Thinking Sprint 11 Is Full SQL Practice

Sprint 11 is not mainly about writing many SQL queries.

Sprint 11 is about:

- relational database ideas
- PostgreSQL setup
- tool understanding
- basic connection practice

SQL commands start properly in Sprint 12.

Mistake 5: Deleting the Data Directory Carelessly

Problem: You may damage your PostgreSQL data.

Better habit: Understand that data files live in the data directory.

Mistake 6: Ignoring the Port Setting

Problem: You forget which port PostgreSQL uses.

Better habit: Remember PostgreSQL usually uses `5432`.

42. Sprint 11 Checkpoint

You pass Sprint 11 when you can:

- open pgAdmin
- open psql
- connect to the default `postgres` database
- explain relational vs non-relational databases in your own words
- recognize the PostgreSQL prompt

Required checkpoint:

```
psql -U postgres -d postgres
```

Expected prompt:

```
postgres=#
```

Simple meaning: If you can open psql and recognize the prompt, PostgreSQL is working.

43. Good Beginner Checkpoint Answer

A good simple explanation is:

A relational database stores data in tables with rows and columns. PostgreSQL is a relational database system. pgAdmin is the visual tool, and psql is the terminal tool. If I see `postgres=#`, I am connected to the default PostgreSQL database.

44. Mini Assessment Before Sprint 12

Answer these questions before moving forward:

1. What is a relational database?
2. What is a table?
3. What is a row?
4. What is a column?
5. What is a record?
6. What is an attribute or field?
7. What is a schema?
8. What is a query?
9. What is a primary key?
10. What is a foreign key?
11. What is the difference between relational and non-relational databases?
12. Name some common relational database systems.
13. What is PostgreSQL?
14. What is the difference between SQL and PostgreSQL?
15. What is PostgreSQL Server?
16. What is pgAdmin?
17. What is psql?
18. What is Stack Builder?
19. What is the install directory?
20. What is the data directory?
21. What is the PostgreSQL superuser password?
22. What is the usual PostgreSQL port?
23. What does locale mean in PostgreSQL setup?
24. What is the default database after PostgreSQL installation?
25. What does `postgres=#` mean?
26. What command checks whether psql is installed?
27. How do you connect to the default database using psql?
28. How do you exit psql?

45. Sprint 11 Completion Checklist

Tick these when completed:

- ☐ I can explain what a relational database is.
- ☐ I can explain table, row, and column in simple words.
- ☐ I understand record, attribute, and field.
- ☐ I can explain what a schema is.
- ☐ I understand the basic idea of primary keys and foreign keys.
- ☐ I can explain relational vs non-relational databases.
- ☐ I know what PostgreSQL is.
- ☐ I know the difference between SQL and PostgreSQL.

- ☐ I know what PostgreSQL Server does.
 - ☐ I know what pgAdmin is.
 - ☐ I know what psql is.
 - ☐ I know what Stack Builder is.
 - ☐ I understand install directory and data directory.
 - ☐ I remember that PostgreSQL usually uses port `5432`.
 - ☐ I understand what locale means at a beginner level.
 - ☐ I can check `psql --version`.
 - ☐ I can run `psql -U postgres -d postgres`.
 - ☐ I can recognize the `postgres=#` prompt.
 - ☐ I can exit psql using `\q`.
 - ☐ I am ready to move to Sprint 12.
-

46. Final Simple Meaning of Sprint 11

Sprint 11 teaches you the basic ideas behind relational databases and prepares your PostgreSQL tools.

You are not deeply writing SQL yet.

You are learning:

- what the database is
- how the data is organized
- what the PostgreSQL tools do
- how to connect successfully

Simple final meaning: Sprint 11 is the bridge between Git learning and real database learning.